ТЕМА 6. ДОПОЛНИТЕЛЬНАЯ ПРАКТИКА

ЛЕКЦИЯ 6.11. НОВЫЕ ВОЗМОЖНОСТИ CSS.

Вложенность. Цветовые пространства. Трансформации. Запросы к контейнерам. Медиа-запросы. Функции. Настройка текста.

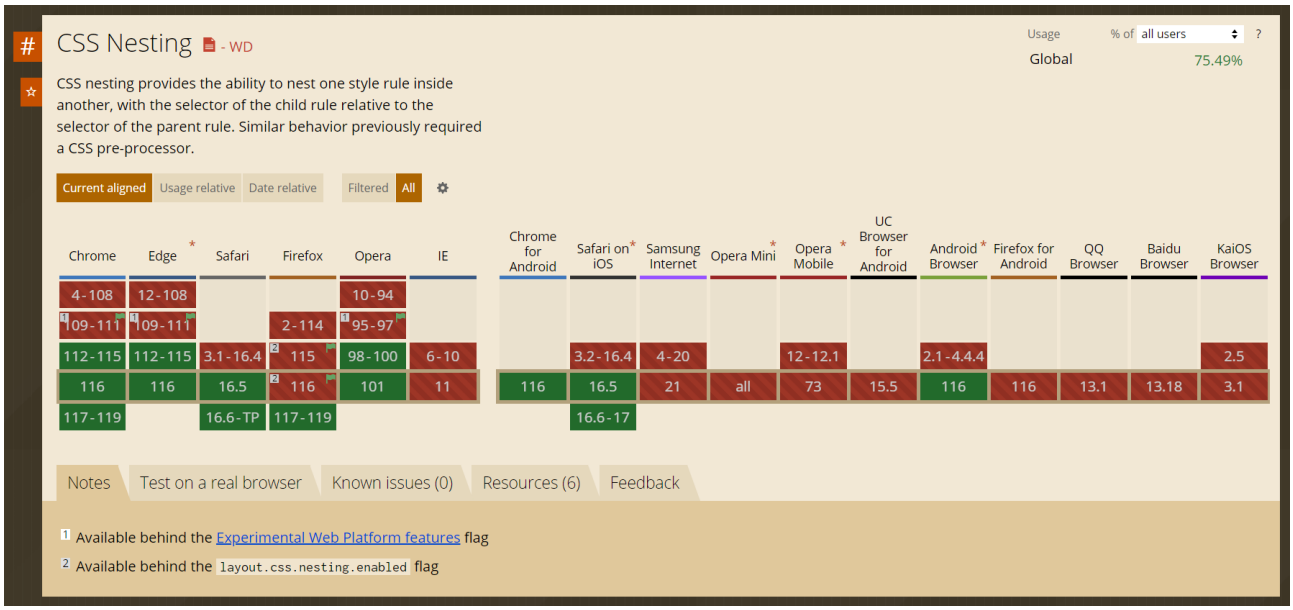
Общие правила CSS

Вложенность

В прошлой лекции поговорили о переменных, которые позволяют упорядочить внесение изменений в код, сделать его более понятным. Еще один способ сделать наш код лучше – вложенность.

Вложение CSS экономит время набора текста и может упростить чтение и поддержку синтаксиса. А еще позволяет разработчикам писать код в более сжатом, сгруппированном виде, что уменьшает избыточность кода.

До апреля 2023 года ни один браузер не понимал вложенный синтаксис CSS. На август 2023 поддержка возможности выглядит следующим образом.



©Pechora-PRO

Также, при работе с вложенностью рекомендуется следовать некоторым правилам. С окончательным утверждением стандарта, возможно некоторые из них станут не нужны.

Вложение CSS позволяет определять стили для элемента в контексте другого селектора.

Вы можете вложить любой селектор в другой, но он должен начинаться с символа, такого как **&** (указатель текущего селектора), **.** (для class), **#** (для id), @ (для медиа-запроса), **:**, **::**, ***, **+**, ~, **>** или **[**. Другими словами, это не может быть прямая ссылка на HTML элемент. Например, код ниже недействителен, и селектор <p> не анализируется:

```
.parent {
    p {
      color: blue;
    }
}
```

Самый простой способ исправить ошибку — использовать амперсанд (&), который ссылается на текущий селектор:

```
.parent {
  & p {
    color: blue;
  }
}
```

Еще один пример.

Без вложенности стили для элементов с классом **button**, находящихся внутри блоков **.card** будет выглядеть следующим образом:

```
.card .button {
  background-color: #c0c0c0;
  border: 1px solid blue;
}
```


©Pechora-PRO

С вложенностью, есть два допустимых способа:

```
/* & явно помещается перед .button */
.card {
  & .button {
    background-color: #c0c0c0;
    border: 1px solid blue;
  }
}
```

или

```
/* без &, только с отступом */
.card {
  .button {
    background-color: #c0c0c0;
    border: 1px solid blue;
  }
}
```

Также & позволяет нацеливать псевдоэлементы и псевдоклассы на родительский селектор. Например:

```
p.my-element {
    &::after {...}
    &:hover {...}
    &:target {...}
}
```

Обратите внимание, что вы можете использовать в & любом месте селектора. Например:

```
.child {
    .parent & {
      color: red;
    }
  }
```

Эта запись аналогична следующей:

```
.parent .child { color: red; }
```

Также вложенными могут быть и media-запросы.

```
.card {
    font-size: 1rem;

    @media (width >= 1024px) {
```


©Pechora-PRO

```
      &.large {
        font-size: 1.25rem;
      }
    }
  }
```

Цветовые пространства с широкой гаммой

В одной из лекций мы уже говорили о способах записи цветов, но для сегодняшней темы важным здесь является не форма представления, а диапазон возможных значений.

Ни для кого не секрет, что человеческий глаз воспринимает гораздо большее количество цветов, чем самая совершенная камера. Именно поэтому, потрясающий закат на снимке может выглядеть гораздо хуже, чем на самом деле.

Но, если средства съемки и дисплеи совершенствуются, а значит мы можем добавить на свою страницу изображение высокого качества и с большим количеством цветов.

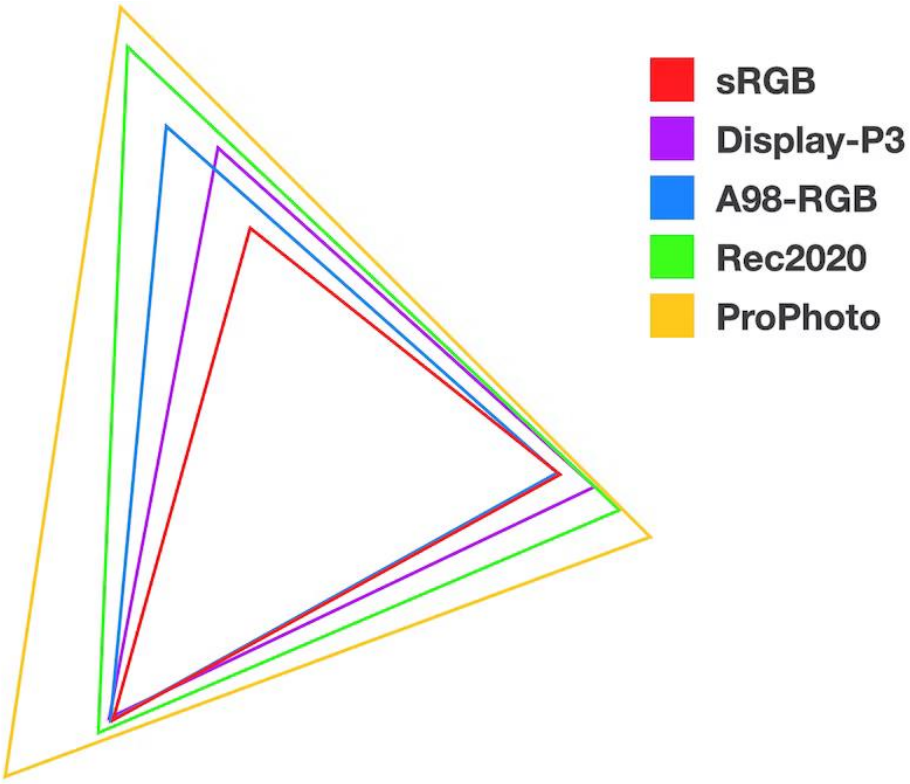
Но, к сожалению, еще недавно создать элемент интерфейса или градиент или … (вставьте нужное)… в цветах из этих же диапазонов мы не могли.

Теперь у нас есть диапазон новых цветовых пространств, включая REC2020, P3, XYZ, LAB, OKLAB, LCH и OKLCH.

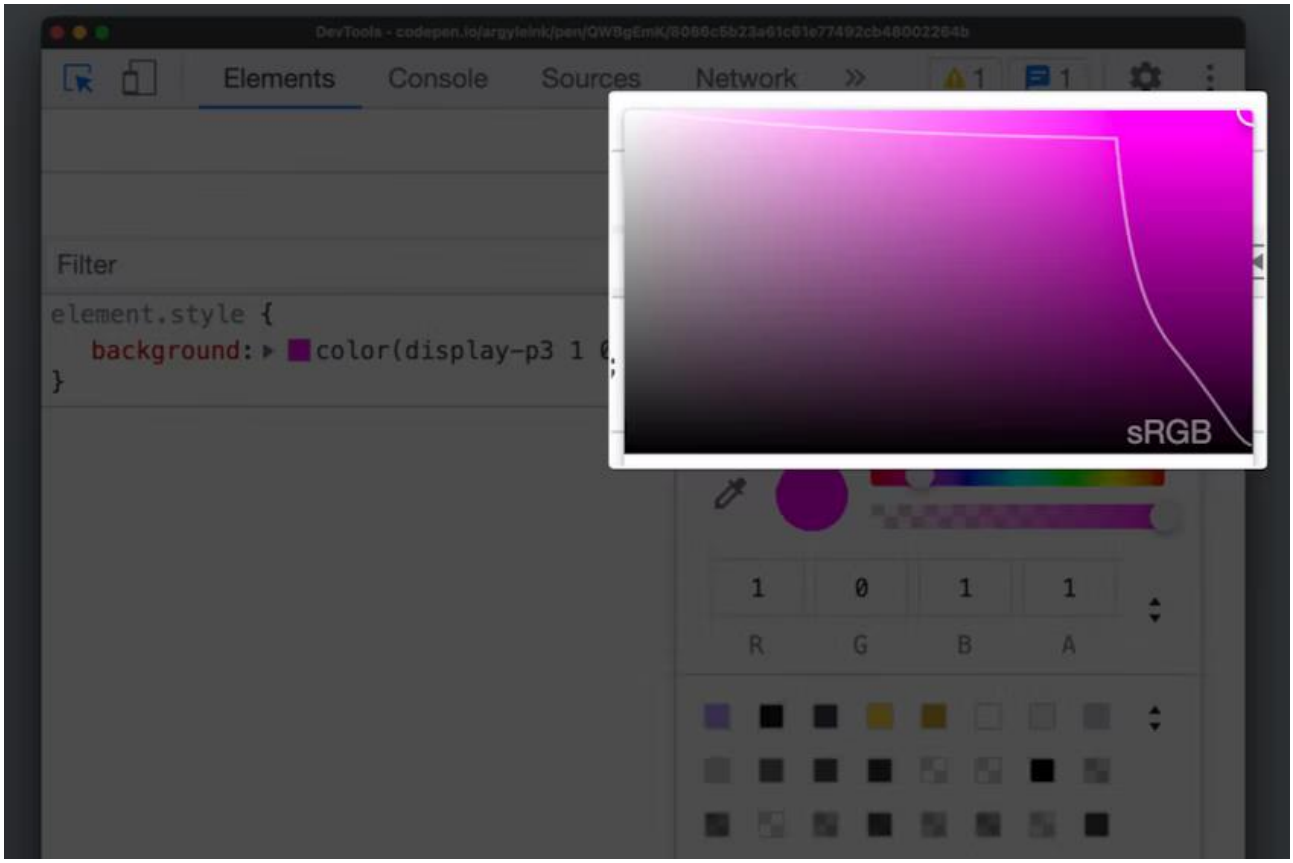
Введение новых цветовых пространств позволило расширить диапазон используемых цветов во много-много раз.

Наглядную разницу можно оценить здесь - https://codepen.io/meodai/pen/zdgXJj

©Pechora-PRO



На изображении ниже белой линией отмечена граница между диапазоном srgb и диапазоном пространств с широкой гаммой.



©Pechora-PRO

Подробную информацию о новых цветовых пространствах можно посмотреть здесь.

Обратиться к этим пространствам можно с помощью функции **color()**. Она позволяет указать цвет в определенном цветовом пространстве, а не в неявном цветовом пространстве sRGB, в котором работает большинство других функций цвета.

Общий вид: **color(colorspace c1 c2 c3 [/A])**

colorspace

Обозначает одно из предопределенных цветовых пространств: srgb, srgb-linear, display-p3, a98-rgb, prophoto-rgb, rec2020, xyz, xyz-d50, xyz-d65.

c1, c2, c3

числовое или процентное значение или ключевое слово **none**, которые предоставляют значения компонентов в цветовом пространстве.

A *необязательный элемент*

An числовое значение или ключевое слово **none** для обозначения прозрачности, где число 1 соответствует 100% (полная непрозрачность).

Индивидуальные свойства трансформации

В теме 4 Анимация в CSS мы познакомились со свойством transform. С его помощью можно масштабировать (scale), вращать (rotate) или смещать (translate) элементы на странице.

При этом, если при взаимодействии с элементом нам было необходимо в определенный момент времени изменить одну из настроек, то переписывать приходилось все значения.

```
.target {
```


©Pechora-PRO

```
  transform: translateX(50%) rotate(30deg) scale(1.2);
}

/* Изменился только масштаб, но мы должны повторить все
функции */
.target:hover {
  transform: translateX(50%) rotate(30deg) scale(2);
}
```

Теперь трансформации можно разделять и применять по отдельности. Для этого настройки задаем не через значения свойства transform, а с помощью отдельных свойств на каждую из трансформаций.

```
.target {
  translate: 50% 0;
  rotate: 30deg;
  scale: 1.2;
}

.target:hover {
  scale: 2;
}
```

Одним из ключевых различий между исходным **transform** свойством и новыми является порядок применения объявленных преобразований.

При использовании функции **transform**() преобразования применяются в том порядке, в котором они записаны – **слева направо**.

Порядок отдельных свойств преобразования не совпадает с порядком их объявления. **Порядок всегда один и тот же**: сначала translate, затем rotate, а затем scale.

Если одно из отдельных свойств преобразования объявлено вместе с transform(), то сначала применяются отдельные

©Pechora-PRO

преобразования (translate, rotate, scale), а transform() последними.

Медиа-запросы

Запросы к контейнерам

Свойство получило поддержку относительно недавно, поэтому оставим его пока здесь, к тому же некоторые его расширенные возможности до сих пор только некоторым браузерам.

Запросы к контейнерам — функция в CSS, которая позволяет адаптировать стили в зависимости от размера их контейнера, а не области просмотра. Это обеспечивает большую гибкость при создании отзывчивого дизайна для различных компонентов на странице.

С помощью запросов к контейнерам можно создавать более модульные и многократно используемые компоненты, заключая их стили в контейнер, что обеспечивает им большую автономность и меньшую степень зависимости от контекста страницы.

Синтаксис запросов к контейнерам похож на синтаксис медиа-запросов, но с небольшим отличием. Вместо **@media**, для определения запроса к контейнеру нужно использовать **@container**. Вот наглядный пример:

```
 .container {
/* Стили, применяемые к самому контейнеру */
    container-type: inline-size;

 }
 @container (min-width: 300px) {
   .container {
/* Стили, применяемые при ширине контейнера не менее 300px */
   }
 }
 @container (min-width: 600px) {
```


©Pechora-PRO

```
    .container {
/* Стили, применяемые при ширине контейнера не менее 600px */
    }
 }
```

В отличие от традиционных медиа-запросов, которые реагируют на область просмотра, запросы к контейнерам реагируют на изменения размера элемента, который их содержит. Таким образом, с помощью запросов к контейнерам можно без проблем стилизовать компоненты по отдельности.

Для определения состояния контейнера можно использовать логические ключевые слова:

- **and** объединяет два или более условий.
- **or** объединяет два или более условий.
- **not** отменяет условие. Для каждого запроса контейнера допускается только одно условие «не», и его нельзя использовать с ключевыми словами and или or.

Параметры контейнера настраиваются с помощью свойств

- **container-name –** имя контейнера (или список имен) для применения стилей. Важно чтобы имя не было записано в кавычках. При этом можно использовать имена идентификаторов, определенных автором документа (например, --container-name).
- **container-type**
  - **size** устанавливает контейнер запросов для запросов размера контейнера как на линейной, так и на блочной оси как во встроенном, так и в блочном измерении. Применяет к контейнеру ограничение макета, стиля и размера.
  - **inline-size** устанавливает контейнер запросов на встроенной оси контейнера. Применяет к элементу макет, стиль и ограничение встроенного размера.

©Pechora-PRO

- **normal** этот элемент не является контейнером запросов для запросов любого размера контейнера, но остается контейнером запросов для запросов стиля контейнера.

или их обобщения **container**.

Их указание в качестве параметров **@container** позволяет конкретизировать использование правила.

В условии контейнера можно использовать следующие дескрипторы:

- **aspect-ratio** отношение ширины контейнера к высоте контейнера, выраженное в виде значения.
- **block-size** размер по второстепенной оси.
- **height** высота контейнера.
- **inline-size** размер по главной оси.
- **orientation** ориентация контейнера: landscape или portrait.
- **width** ширина контейнера.

При применении стилей к контейнеру с помощью контейнерных запросов вы можете использовать **единицы длины запроса контейнера**. Эти единицы определяют длину относительно размеров контейнера запроса. Компоненты, использующие единицы длины относительно своего контейнера, более гибки для использования в разных контейнерах без необходимости пересчета конкретных значений длины.

Единицами длины запроса контейнера являются:

- **cqw**: 1 % ширины контейнера запроса.
- **cqh**: 1% высоты контейнера запроса.
- **cqi**: 1 % от встроенного размера контейнера запроса.
- **cqb**: 1 % от размера блока контейнера запроса.
- **cqmin**: Меньшее значение либо cqi или cqb

©Pechora-PRO

- **cqmax**: большее значение либо cqi или cqb

Запросы стиля

Спецификация запросов контейнера также позволяет запрашивать значения стилей родительского элемента. В настоящее время данная возможность **частично реализована** в браузерах, где для применения стилей контейнера можно использовать переменные CSS.

В следующем примере используется значение доступности, хранящееся в переменной, для стилизации фона карточки. Если в стилях элемента он обозначен как доступный, ему добавляется фон.

```
<div class="container-card " style="--success: true;">
    ...
</div>
```

```
@container style(--success: true) {
  .container-card {
    background: linear-gradient(-30deg, yellow, orange);
  }
 }
```

Интересно, что если в качестве проверяемого значения используется цвет, то способ его указания должен быть одинаковым. Например, ключевое слово **blue** и эквивалентный ему шестнадцатеричный код **#0000ff** не будут соответствовать друг другу.

Планируется добавление к возможностям определения стилей использование логических запросов, диапазонов значений и т.д.

Функции

Тригонометрические функции

©Pechora-PRO

По мере развития CSS в него были включены возможности написания математических выражений. Мы рассмтаривали функцию calc()для выполненичя вычисления, и другие полезные функции, такие как min(), max() и clamp().

Новые функции, определенные в модуле CSS Values и Units Module Level 4, позволяют разработчикам выполнять тригонометрические вычисления непосредственно в CSS, открывая новые возможности для создания креативного и динамичного дизайна: sin(), cos(), tan(), asin(), acos(), atan() и atan2().

На сегодня использование тригонометрических функций поддерживается всеми основными браузерами кроме Opera.

color-mix()

Функция **color-mix** позволяет смешивать значения двух цветов для создания новых значений на основе каналов смешиваемых цветов. На результат влияет пространство цветов, в котором осуществляется смешивание.

```
color-mix(in srgb, red, white);
color-mix(in srgb-linear, red, white);
color-mix(in lch, red, white);
color-mix(in oklch, red, white);
color-mix(in lab, red, white);
color-mix(in oklab, red, white);
color-mix(in xyz, red, white);
```

Функция предоставляет возможность сохранять непрозрачные значения цвета при добавлении к ним некоторой прозрачности. Можно использовать переменные с цветами бренда для создания вариаций цвета с разной прозрачностью. Это делается с помощью смешения цвета с прозрачностью. Например, когда мы смешиваем красный цвет с 10% прозрачностью, то получаем 90%-непрозрачный красный цвет.

©Pechora-PRO

```
color-mix(in srgb, red, transparent 10%);
```

Это позволяет легко и быстро создавать большие цветовые системы.

Дополнительно после выбора цветового пространства можно указать метод интерполяции (выбора промежуточных значений) цветов: shorter hue, longer hue, increasing hue, decreasing hue.

Проверить эффект от смешение цветов в разных пространствах можно с помощью сервиса **https://color-mix.style/**

Псевдоклассы и псевдоэлементы

:has()

Псевдокласс **:has()** является, вероятно одним из мощнейших дополнений к селекторному механизму CSS. Этот псевдокласс представляет способ выбора родительского элемента или предыдущего родственного элемента относительно ссылочного элемента, принимая в качестве аргумента относительный список селекторов.

```
div:has(ul.blue) {
  background-color: yellow;
}
```

Фоновый цвет, применяется к элементам **div**, которые содержат элемент **ul.blue** на любом уровне вложенности.

Аналогичный код, но с указателем **>** ограничит уровень вложенности первым.

```
div:has(> ul.blue) {
  background-color: yellow;
}
```

С помощью комбинатора **+** можно задать расположение вложенного элемента непосредственно после родительского.

```
h1:has(+ h2) {
  margin: 0 0 0.25rem 0;
```


©Pechora-PRO

```
}
```

Например, задаем заголовку первого уровня дополнительный отступ, если непосредственно после него расположен заголовок второго уровня.

:has() можно использовать для проверки истинности одной из нескольких функций или истинности всех функций.

Используя значения, разделенные запятыми, внутри **:has()**, вы проверяете, существует ли какой-либо из параметров.

x:has(a, b)

*будет стилизовать **x**, если у него существует потомок **a или b**.*

Объединяя несколько **:has()**, вы проверяете, существуют ли все параметры.

x:has(a):has(b)

*будет стилизовать **x**, если существуют **потомок a и потомок b**.*

Также, используя комбинацию псевдоклассов **:has** и **:is** можно описать несколько вариантов сочетаний элементов.

```
:is(h1, h2, h3):has(+ :is(h2, h3, h4)) {
  margin: 0 0 0.25rem 0;
}
```

Указанные отступы будут применятся ко всем заголовкам соседним с заголовками.

Или объедините несколько условий и выберите **article**, если за заголовком следует подзаголовок.

```
article:has(> h1 + h2) { … }
```

nth-of

Продвинутый синтаксис **nth-child** позволяет использовать ключевое слово **of** для ограничения существующего микросинтаксиса An + B определенным набором элементов.

©Pechora-PRO

При использовании обычного nth-child, например, :nth-child(2) на классе special, браузер выберет элемент с классом special, если он является вторым потомком.

:nth-child(2 of .special) сначала отфильтрует все элементы с классом special, затем возьмет второй элемент из этого списка.

Настройка текста

text-wrap: balance

При работе с текстом в заголовках и подзаголовках слова по строкам часто распределяются не очень удачно. Чтобы обеспечить соответствие макетам или улучшить внешний вид заголовка приходилось манипулировать размерами оберток или добавлять разрывы строк вручную. Все это не очень удобно, да и не очень правильно.

Теперь вы можете настроить автоматически сбалансированный перенос текста для заголовков с помощью свойства **text-wrap** со значением **balance**.

Для балансировки текста браузер выполняет эффективный бинарный поиск наименьшей ширины, которая не вызывает переноса строки, останавливаясь на каждом CSS пикселе (не экранном). Для дальнейшей оптимизации бинарного поиска браузер начинает с 80% средней ширины строки.

Такая возможность работает только до 4 строк текста, поэтому он отлично подходит для заголовков и подзаголовков, но не подходит для более длинного контента.

initial-letter

Другим приятным улучшением веб-типографии является свойство **initial-letter**.

©Pechora-PRO

initial-letter используется на псевдоэлементе **:first-letter** для определения размера первой буквы на основе того, сколько строк она занимает и на сколько смещается относительно своего стартового положения.

Ключевое слово **normal** в качестве значения не задает никакого особого эффекта начальной буквы. Текст ведет себя как обычно.

Первое число - определяет размер начальной буквы в зависимости от того, сколько строк она занимает. Отрицательные значения не допускаются.

Второе число - определяет количество строк, на которые должна утонуть начальная буква, если задан ее размер. Значения должны быть больше нуля. Если этот параметр не указан, он дублирует значение размера, округляя его до ближайшего положительного целого числа.

Оба числа записываются в качестве значения свойства через пробел.

©Pechora-PRO